

Resolución: Coloreo Mínimo de Grafos mediante Backtracking

Este documento presenta la resolución detallada para encontrar el número cromático $\chi(G)$ de un grafo no dirigido $G = (V, E)$ usando **Backtracking** (Vuelta Atrás). El objetivo es asignar a cada vértice un color de tal manera que ningún par de vértices conectados compartan el mismo color, minimizando la cantidad total de colores empleados.

a) Representación de Soluciones y Restricciones

1. Representación de las Soluciones

Asumimos que el grafo tiene n vértices indexados de 1 a n , es decir, $V = \{v_1, v_2, \dots, v_n\}$, y un conjunto de aristas E . Representamos la coloración como una tupla de tamaño n :

$$X = (x_1, x_2, \dots, x_n)$$

Donde cada componente x_i representa el identificador numérico del color asignado al vértice v_i .

2. Restricciones Explícitas

En el peor de los casos (un grafo completo K_n), necesitaremos a lo sumo n colores diferentes. Por lo tanto, el dominio de cada variable es:

$$x_i \in \{1, 2, \dots, n\} \quad \forall i \in \{1, \dots, n\}$$

3. Restricciones Implícitas

Las restricciones implícitas que garantizan una solución válida y óptima son:

1. **Adyacencia (Restricción de Coloreo Válido):** Dos vértices conectados por una arista no pueden compartir el mismo color.

$$\forall (v_i, v_j) \in E \implies x_i \neq x_j$$

2. **Minimización (Restricción de Optimalidad):** Si definimos la cantidad de colores únicos usados en la tupla X como $C(X) = |\{x_1, \dots, x_n\}|$, buscamos minimizar este valor respecto a cualquier otra asignación válida:

$$\chi(G) = \min C(X)$$

Durante la ejecución, mantendremos una variable global $mejor_G$ que almacene la menor cantidad de colores con la que se ha logrado colorear el grafo completo hasta el momento.

b) Árbol del Espacio de Soluciones y su Tamaño

Descripción del Árbol

El espacio de búsqueda se estructura mediante un árbol de decisión n -ario:

- **Raíz (Nivel 0):** Grafo sin colorear. Ninguna decisión tomada.
- **Nivel i ($1 \leq i \leq n$):** Representa la asignación de color al vértice v_i .
- **Ramificación:** Desde un nodo en el nivel $i - 1$, se abren tantas ramas como colores disponibles decidamos probar para el vértice v_i . En un análisis de fuerza bruta sin restricciones, se prueban hasta n colores posibles.
- **Hojas (Nivel n):** Representan asignaciones completas de color para todos los vértices del grafo.

Tamaño del Árbol (Búsqueda Exhaustiva)

Si realizáramos una búsqueda por fuerza bruta evaluando todas las posibilidades de coloreo con hasta n colores:

1. Número de hojas (soluciones candidatas totales): n^n
2. Número total de nodos en el árbol:

$$\sum_{i=0}^n n^i = \frac{n^{n+1} - 1}{n - 1} \approx O(n^n)$$

c) Predicados Acotadores (Pruning)

Para resolver este problema de manera eficiente, el algoritmo de Backtracking debe incorporar tres estrategias de poda críticas para evitar la exploración del árbol completo:

1. Poda por Consistencia Local (Viabilidad Inmediata)

Al evaluar el color c para el vértice v_k , este no puede ser igual al color ya asignado a ninguno de sus vecinos adyacentes que ya hayan sido coloreados en los niveles anteriores.

- **Condición de Poda (Inviabilidad):**

$$\exists j < k \text{ tal que } (v_j, v_k) \in E \quad \wedge \quad x_j = c$$

2. Poda por Cota Superior (Optimización)

Si en el nivel actual k de la recursión ya hemos utilizado una cantidad de colores distintos que es igual o mayor que el mejor mínimo global de colores encontrado previamente ($mejor_C$), podemos podar la rama inmediatamente. Es imposible que esta rama produzca una solución con menos colores.

Sean $U_k = \{x_1, \dots, x_{k-1}\}$ el conjunto de colores ya utilizados en los niveles anteriores.

- **Condición de Poda:**

$$|U_k| \geq \text{mejor_C}$$

d) Algoritmo en Pseudocódigo y Análisis de Coste

A continuación se detalla el diseño del algoritmo recursivo en pseudocódigo para calcular el número mínimo de colores necesarios.

```

Procedimiento CalcularNumeroCromatico(n, E)
Inicio
  // Vector de asignación de colores inicializado en 0 (sin colorear)
  color_asignado <- Arreglo de tamaño n + 1 inicializado en [0, ..., 0]

  // El peor caso inicial para colorear un grafo de n vértices son n colores
  mejor_C <- n

  // Función auxiliar para verificar si un color es válido para el vértice actu
  Funcion EsSeguro(vertice, color_a_probar)
  Inicio
    Para vecino <- 1 Hasta n Con Paso 1 Hacer
      Si (vecino, vertice) en E y color_asignado[vecino] = color_a_probar E
        Retornar Falso
      FinSi
    FinPara
    Retornar Verdadero
  FinFuncion

  // Procedimiento recursivo de backtracking
  Procedimiento Colorear(vertice, colores_usados)
  Inicio
    // 1. PODA POR COTA SUPERIOR: Si ya igualamos o superamos el mejor C encc
    Si colores_usados >= mejor_C Entonces
      Retornar
    FinSi

    // 2. CASO BASE: Todos los vértices han sido coloreados con éxito
    Si vertice = n + 1 Entonces
      Si colores_usados < mejor_C Entonces
        mejor_C <- colores_usados
      FinSi
      Retornar
    FinSi

    // 3. RAMIFICACIÓN Y EXPLORACIÓN (Aplicando ruptura de simetrías)
    // Intentar asignar colores ya usados
    Para c <- 1 Hasta colores_usados Con Paso 1 Hacer
      Si EsSeguro(vertice, c) Entonces
        color_asignado[vertice] <- c
        Colorear(vertice + 1, colores_usados)
        color_asignado[vertice] <- 0 // deshacer para backtracking
      FinSi
    FinPara

    // Intentar asignar un color nuevo (colores_usados + 1)
    Si EsSeguro(vertice, colores_usados + 1) Entonces
      color_asignado[vertice] <- colores_usados + 1
      Colorear(vertice + 1, colores_usados + 1)
      color_asignado[vertice] <- 0 // deshacer para backtracking
  FinFuncion

```

```
        FinSi
    FinProcedimiento

    // Iniciar el algoritmo con el vértice 1 y 0 colores usados
    Colorear(1, 0)

    Imprimir("El número mínimo de colores necesarios (número cromático) es: ", me
FinProcedimiento
```

Análisis de Coste

- **Complejidad Temporal:** $O(n^n)$ en el peor caso teórico sin podas, reduciéndose drásticamente en la práctica a $O(B^n)$ (donde $B \ll n$) gracias a la ruptura de simetrías por isomorfismo de colores y la acotación por cota superior.
- **Complejidad Espacial:** $O(n)$ debido a la profundidad máxima de la pila recursiva (n niveles de llamadas) y al almacenamiento del vector de colores de tamaño n .